

ACROPOLIS INSTITUTE OF TECHNOLOGY AND RESEARCH

Department of IT, CSE(DS)

Synopsis On

TECHPREP AI: INTELLIGENT AI INTERVIEW PRACTICE PLATFORM

1. Introduction

1.1 Overview

In the contemporary technical recruitment landscape, candidates face high performance bars spanning technical execution, system design knowledge, and behavioral articulation. Traditional preparation platforms provide static practice sets (e.g., coding problems on LeetCode or static list questionnaires) but fail to replicate the dynamic, conversational, and multi-disciplinary experience of a live human interview.

TechPrep AI is an intelligent, low-latency, full-stack mock interview platform designed to address these gaps. The system integrates real-time browser-based voice interaction, automated proctoring assessments, and domain-locked adaptive question sequencing. Utilizing a decoupled three-tier architecture, the platform comprises a responsive React client, a high-throughput Fastify API gateway, and a cognitive FastAPI service powered by state-of-the-art Large Language Models (Gemini 2.0 Flash and Llama 3.1 via Groq). During an active session, the candidate is prompted verbally via Text-to-Speech (TTS), responds verbally through automated Speech-to-Text (STT) transcription, and is monitored for behavioral integrity using a client-side computer vision model (TensorFlow.js BlazeFace) that flags look-away, multiple faces, and motion violations.

1.2 Purpose

The purpose of TechPrep AI is to democratize high-fidelity placement preparation and interview coaching. The key goals achieved by the platform include:

- **Dynamic Conversational Practice:** Bridges the gap between static text inputs and verbal articulation by forcing candidates to formulate answers aloud within structured timing windows.
- **Objective, Metric-Based Feedback:** Evaluates student responses across a comprehensive, industrial grading rubric (STAR structure, technical depth, communication, relevance, confidence, and conciseness) to produce quantitative scores out of 100.
- **Low-Cost Integrity Verification:** Provides lightweight, client-side, browser-native proctoring that detects visual cheating anomalies without requiring intrusive kernel-level desktop installations.
- **Institutional Placement Analytics:** Empowers university placement coordinators and recruiters to track cohort performance, identify skill gaps, and review automated PDF candidate reports stored in AWS S3 buckets.

2. Literature Survey

2.1 Existing Problem

Current placement preparation paradigms suffer from three primary limitations. Traditional coding sites evaluate syntax and test cases but ignore verbal structure, communication quality, and behavioral articulation (e.g., the STAR method). In addition, conversational tools (such as ChatGPT) provide generic, overly positive feedback, fail to enforce domain-locked curricula (often drifting off-topic), and lack chronological session memory constraints. Finally, remote assessments are highly vulnerable to proxy candidates and split-screen note referencing, while existing proctoring software requires heavy downloads that degrade performance on standard student hardware.

2.2 Proposed Solution

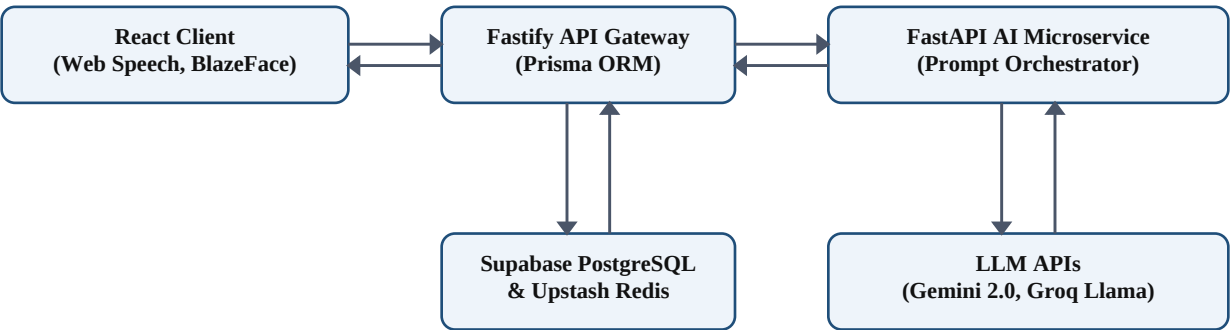
TechPrep AI addresses these issues through a specialized orchestration layer and client-side computer vision:

- Contextual Domain Cycling: A state-machine tracks historical questions, resume keywords, and domain competencies to generate logical, adaptive follow-up questions that push candidates to their technical limits.
- Hybrid Model Execution: Employs low-latency models (gemini-2.0-flash) for responsive, sub-second question generation and high-fidelity models (gemini-1.5-flash or llama-3.1-70b) asynchronously for deeper rubric grading.
- Client-Side Face Tracking: Uses TensorFlow.js BlazeFace locally in the browser. Landmark estimations compute eye-nose coordinates to determine attention vectors, executing proctoring loops at 30 FPS without server overhead.

3. Theoretical Analysis

3.1 Block Diagram

The block diagram below illustrates the structural dependencies, protocol interfaces, and data flow of the TechPrep AI platform:



3.2 Hardware/Software Designing

The requirements needed to develop, run, and host the platform are partitioned below:

Hardware Requirements

- Hosting Server: 1 vCPU, 512 MB RAM, and 10 GB SSD (Render/Vercel virtual environment).
- User Client Device: Dual-Core CPU (Intel i3 / Ryzen 3 or higher), 4 GB RAM, 720p Webcam, and a functional microphone.
- Network Connection: Broadband internet connectivity with a minimum speed of 5 Mbps.

Software Requirements

- Operating Systems: Compatible with Windows 10/11, macOS, and standard Linux distributions.
- Development Runtimes: Node.js (v18.x+) and Python (v3.10.x+).
- Database & Storage: Supabase PostgreSQL relational database and AWS S3 storage buckets.
- Libraries & Frameworks: React.js (v18), Vite, TensorFlow.js (BlazeFace), Prisma ORM, Fastify API Gateway, FastAPI Python Microservice, and ReportLab PDF Compiler.

4. Applications

TechPrep AI provides a versatile utility layer that can be applied across several sectors:

- Academic Placement Cells: Colleges can integrate the platform into placement cells to host mock practice camps, compiling cohort analytics to target critical soft and hard skill deficiencies.
- Corporate Pre-Screening: Human resource departments can deploy the platform to screen candidates' speaking structure and technical accuracy before scheduling synchronous interviews.
- Bootcamps and Ed-Tech: Online training programs can embed this software to prepare students for career transitions by aligning questions directly with custom syllabus modules.
- Self-Paced Practice: Individual job seekers can run mock sessions configured for specific companies, utilizing the resume parser to test their experience against specific job requirements.

REFERENCES

- 1 Google Generative AI Developer Guides: API references and schema enforcement details for Gemini 2.0 Flash (2025).
- 2 Groq Llama 3.1 Inference Guidelines: Low-latency execution, token budgeting, and system instructions optimization (2025).
- 3 TensorFlow.js BlazeFace Model: Bazrev, V. et al. 'BlazeFace: Sub-millisecond Face Detector on Mobile GPUs' (arXiv:1907.05047, 2019).
- 4 ReportLab PDF User Guide: Dynamic document templates, Flowables layout specifications, and vector shapes drawing (2024).
- 5 Fastify Framework Reference: Schema parsing, JSON validation hook sequences, and performance optimization (2024).

Internal Guide:

Prof. Deepak Singh Chouhan
Department of CSE(DS)

External Guide:

Mr./Ms. _____

Group Members:

1. Swayam Garg (Enrollment No. 0111CS221193)
2. _____
3. _____
4. _____